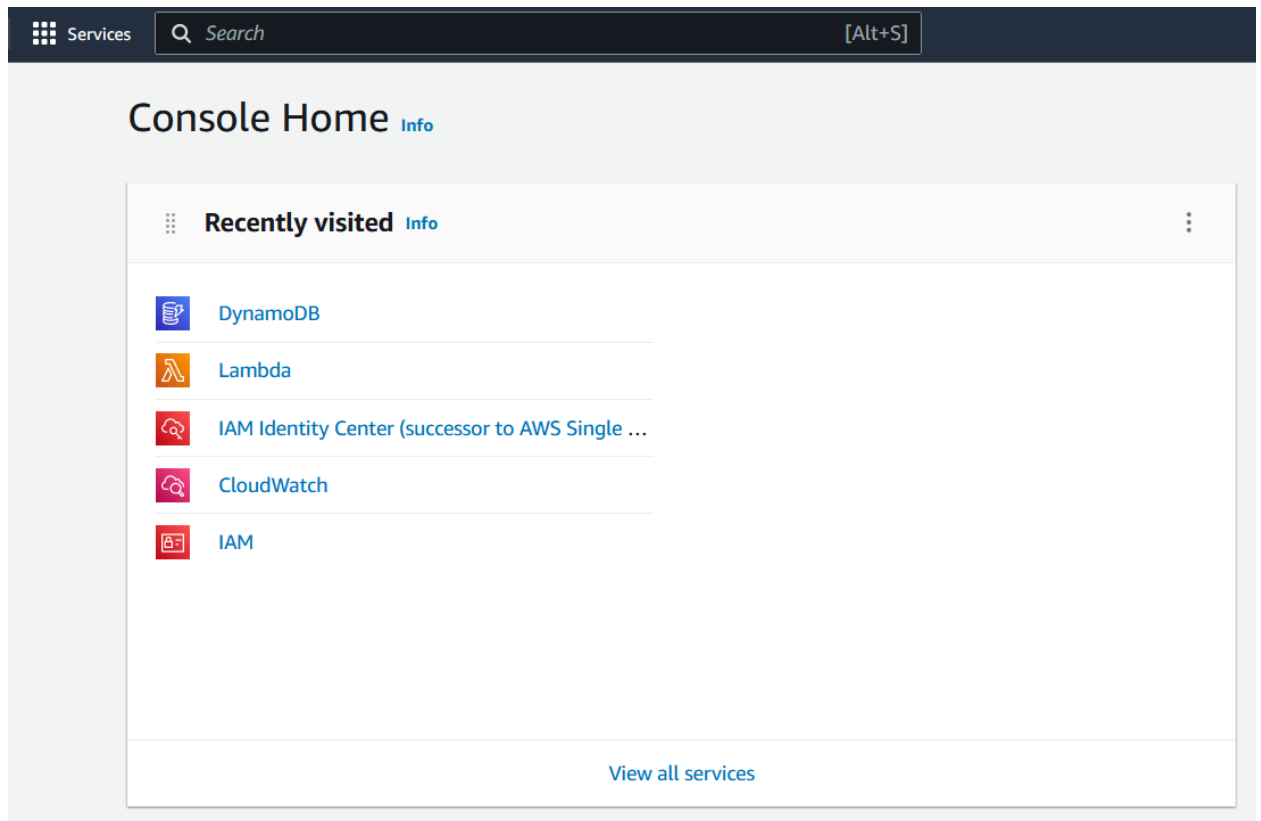
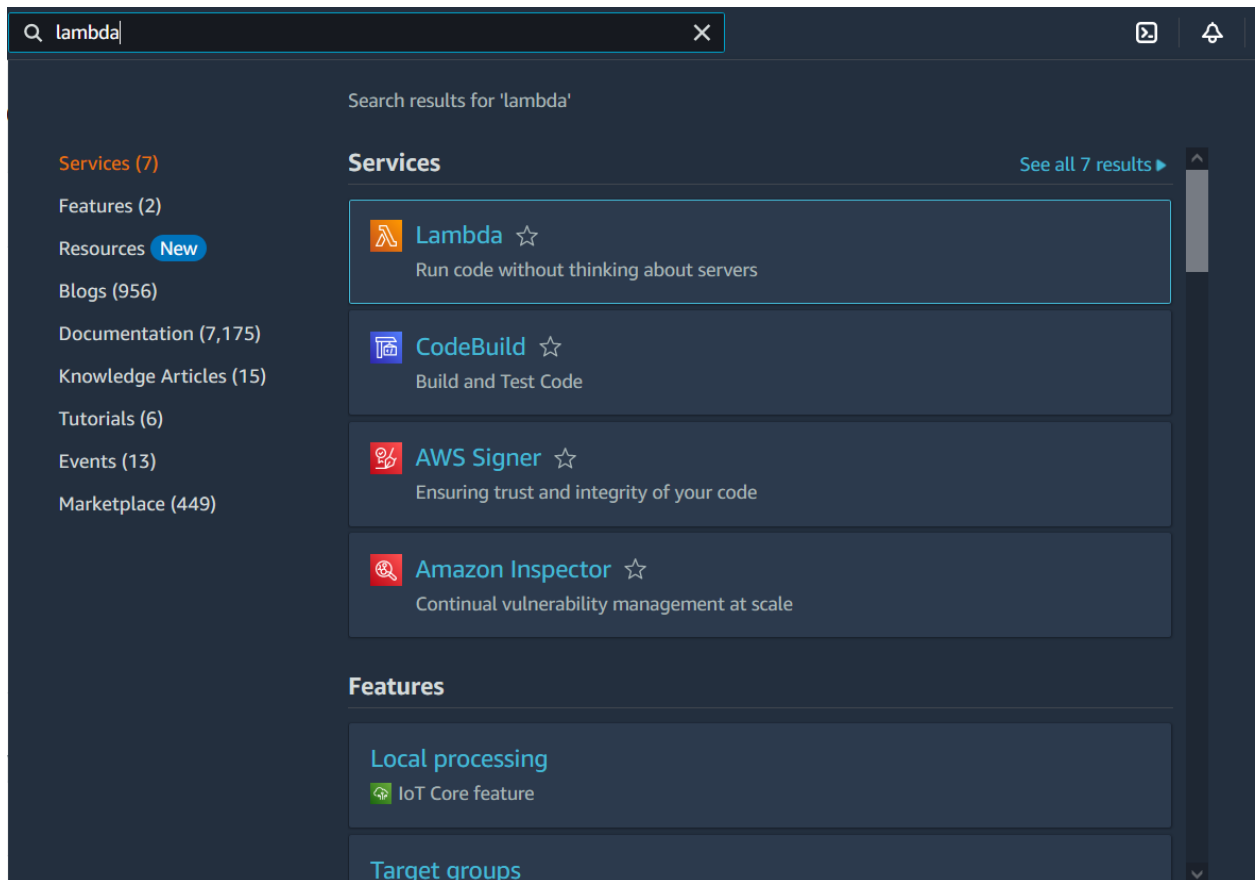


## Accessing the Lambda Functions

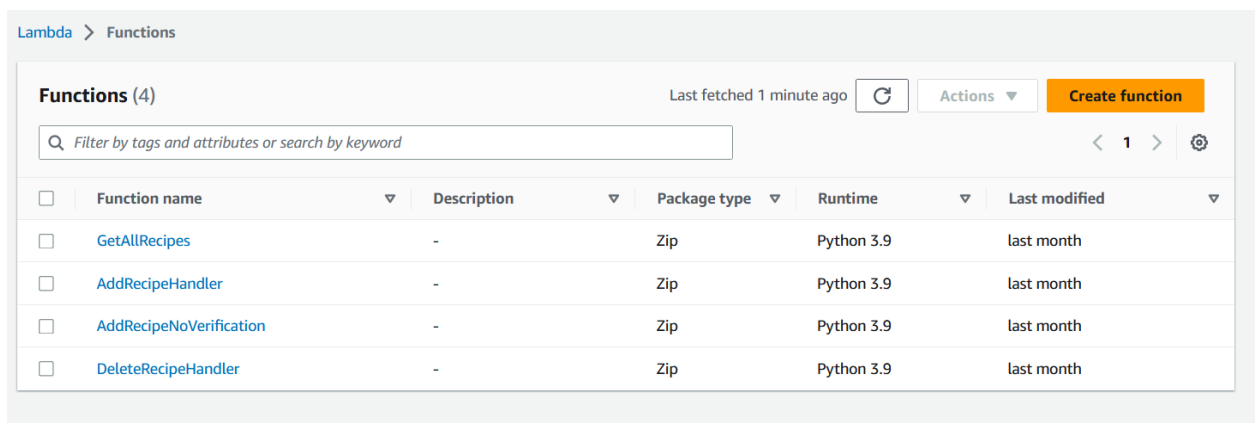
- After signing in, head to the [console](#) on Amazon's server.



- Use the search bar at the top to search for Lambda (If it has been visited recently you can simply click the link in the Recently Visited Section).



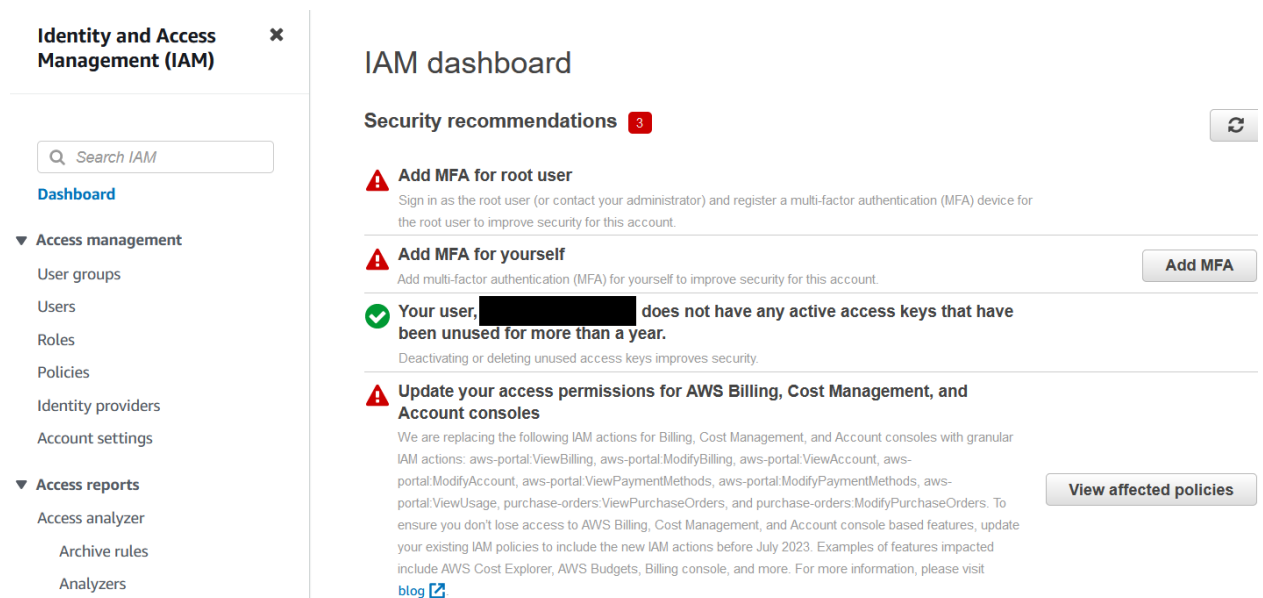
- Clicking on the lambda link will bring you to the list of lambda functions. If there isn't any displayed, make sure that it is using the N. Virginia servers.



- Clicking each function name will open up the relevant information and testing scripts for that specific function.

## Setting up an IDE for Development

- This guide will be using Visual Studio Code for development. Downloads for it can be found [here](#), and the original guide from Amazon can be found [here](#).
- You'll first need to head to the [IAM dashboard](#) to set up an Access Key.



The screenshot shows the AWS IAM dashboard. On the left is a navigation sidebar with the title 'Identity and Access Management (IAM)' and a search bar. The sidebar menu includes 'Dashboard', 'Access management' (with sub-items: User groups, Users, Roles, Policies, Identity providers, Account settings), and 'Access reports' (with sub-items: Access analyzer, Archive rules, Analyzers). The main content area is titled 'IAM dashboard' and features a 'Security recommendations' section with a red badge indicating 3 recommendations. The first recommendation is 'Add MFA for root user' with a red warning icon and a description: 'Sign in as the root user (or contact your administrator) and register a multi-factor authentication (MFA) device for the root user to improve security for this account.' The second recommendation is 'Add MFA for yourself' with a red warning icon and a description: 'Add multi-factor authentication (MFA) for yourself to improve security for this account.' The third recommendation is 'Your user, [redacted] does not have any active access keys that have been unused for more than a year.' with a green checkmark icon and a description: 'Deactivating or deleting unused access keys improves security.' The fourth recommendation is 'Update your access permissions for AWS Billing, Cost Management, and Account consoles' with a red warning icon and a detailed description about replacing IAM actions. A 'View affected policies' button is located at the bottom right of this section.

- Select the Users section on the left side to open the user table. Selecting the user name will open up the permission page for that user.

**Users (5)** [Info](#)

An IAM user is an identity with long-term credentials that is used to interact with AWS in an account.

[Refresh](#) [Delete](#) [Add users](#)

| <input type="checkbox"/> | User name  | Groups                          | Last activity    | MFA  | Password age  | Active key age |
|--------------------------|------------|---------------------------------|------------------|------|---------------|----------------|
| <input type="checkbox"/> | [REDACTED] | <a href="#">RHIT_Developers</a> | ✓ 22 hours ago   | None | ✓ 70 days ago | -              |
| <input type="checkbox"/> | [REDACTED] | <a href="#">RHIT_Developers</a> | ✓ 17 minutes ago | None | ✓ 27 days ago | ✓ 6 days ago   |
| <input type="checkbox"/> | [REDACTED] | None                            | Never            | None | ✓ 70 days ago | -              |
| <input type="checkbox"/> | [REDACTED] | <a href="#">RHIT_Developers</a> | ✓ 2 days ago     | None | ✓ 37 days ago | -              |
| <input type="checkbox"/> | [REDACTED] | <a href="#">RHIT_Developers</a> | Never            | None | ✓ 70 days ago | -              |

### Summary

ARN



arn:aws:iam::238724053954:user/[REDACTED]

Created

February 14, 2023, 17:10 (UTC-05:00)

Console access

⚠ Enabled without MFA

Last console sign-in

✓ Today

Access key 1

AKIATPFI7PBKI3DM2XQ - Active

✓ Used 6 days ago. 6 days old.

Access key 2

Not enabled

[Permissions](#)[Groups \(1\)](#)[Tags \(1\)](#)[Security credentials](#)[Access Advisor](#)

### Permissions policies (4)

Permissions are defined by policies attached to the user directly or through groups.

[Refresh](#) [Remove](#) [Add permissions](#)

**Access keys (1)**

Use access keys to send programmatic calls to AWS from the AWS CLI, AWS Tools for PowerShell, AWS SDKs, or direct AWS API calls. You can have a maximum of two access keys (active or inactive) at a time. [Learn more](#)

[Create access key](#)

 **Local code**

You plan to use this access key to enable application code in a local development environment to access your AWS account.



**Alternative recommended**

Use an Integrated Development Environment (IDE) which supports the AWS Toolkit enabling authentication through IAM Identity Center. [Learn more](#) 

☒ I understand the above recommendation and want to proceed to create an access key.

- Enter a descriptive tag and create the key. Leave the page open or write down the secret key for later use.

**Access key**

If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.

Access key



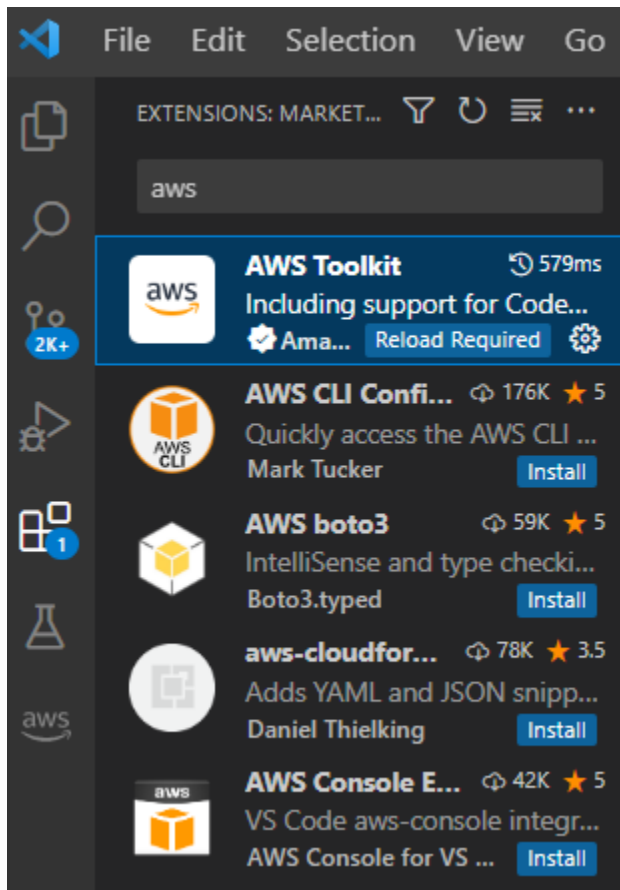
AKIATPFIP7PBHAQM4YOF

Secret access key



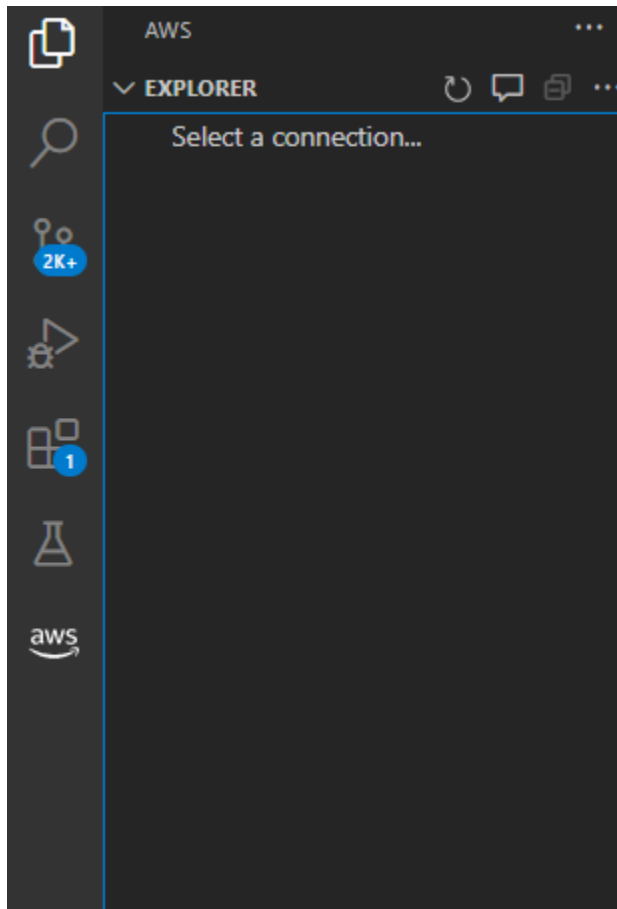
\*\*\*\*\* [Show](#)

- Inside Visual Studio Code, open up the Extensions section and search for AWS Toolkit. Install the extension (Reload may be required).



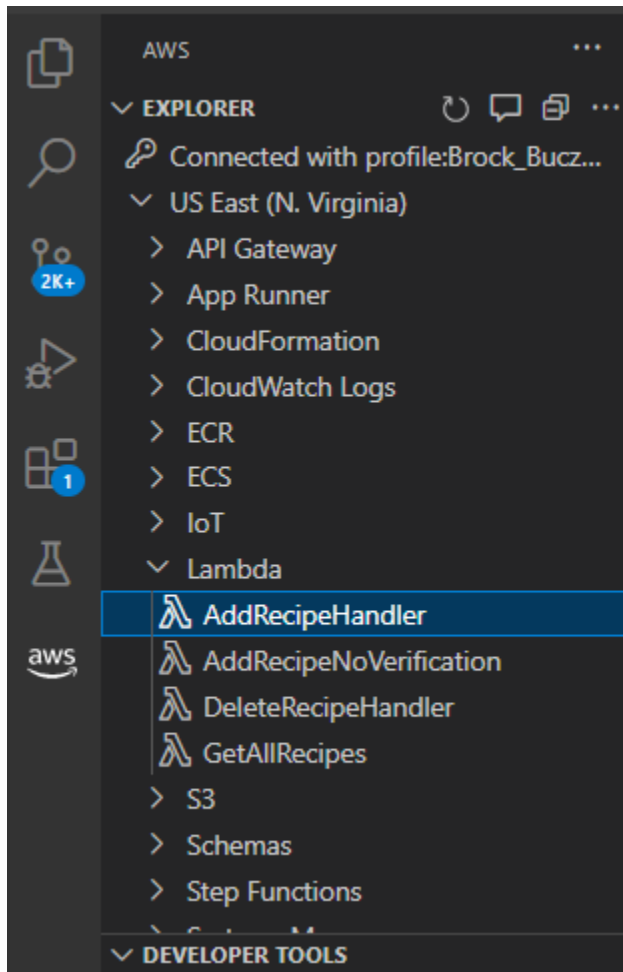
- Select the AWS section and right click the select a connection prompt. When asked to create a new connection, select the option for using IAM Credentials. This will prompt you to put in a name for the connection along with the Access Key and Secret Access Key

that was acquired above.



- After signing in, open the lambda folder to view all the lambda functions. Right click one and choose “download” to download a local copy of the function. You can now open the

file in the IDE to edit the function.

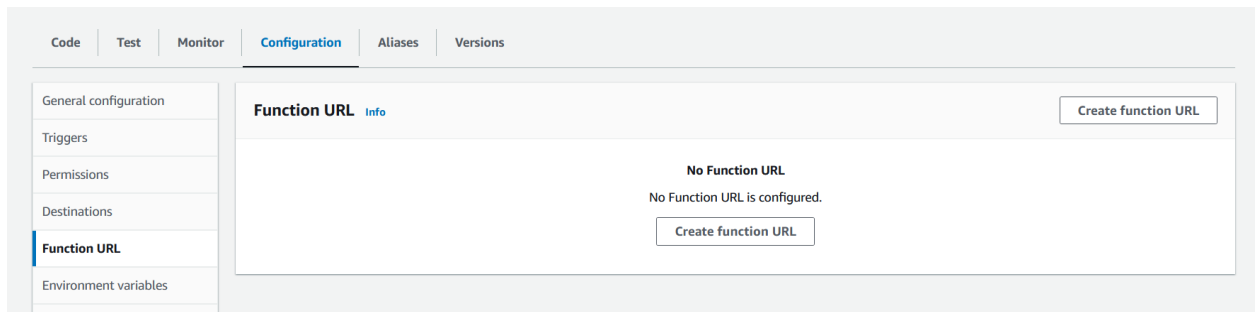


- To upload a function, right click the folder containing the function and select “Upload Lambda”. This will then prompt for the server (N. Virginia) and the function to upload the code to. After confirming, the function will be uploaded to Amazon’s servers.

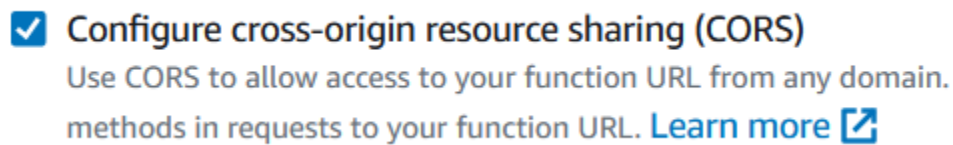
## Letting the App Connect to the Functions



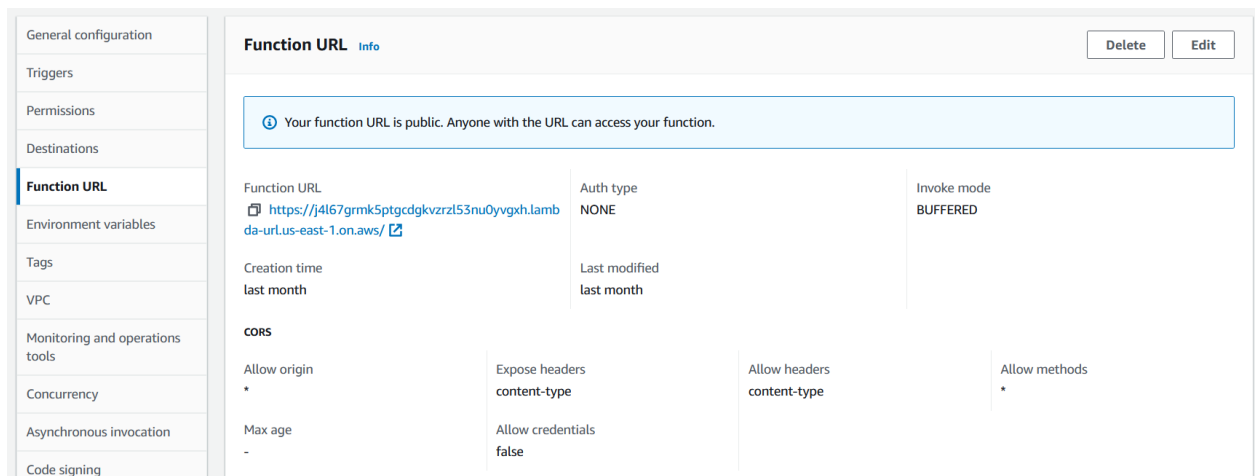
- From the lambda page, select a function and go into the Configuration tab. Then select the Function URL tab.



- Select Create function URL. For the AUTH type select NONE, and then check the option to allow for configuring CORS.



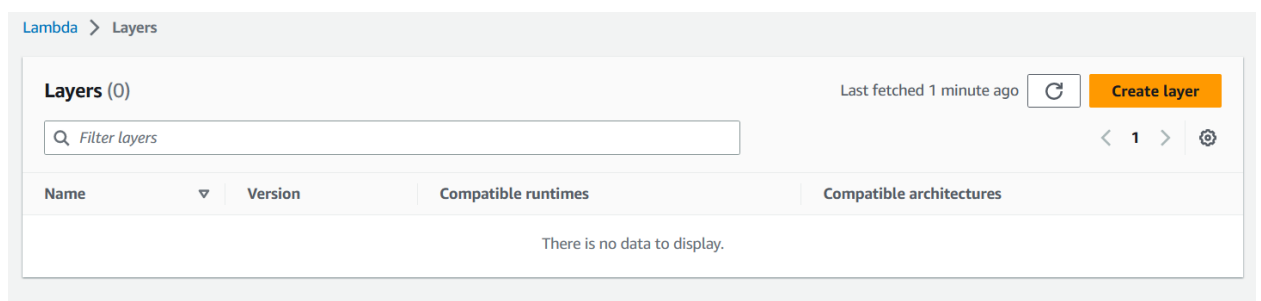
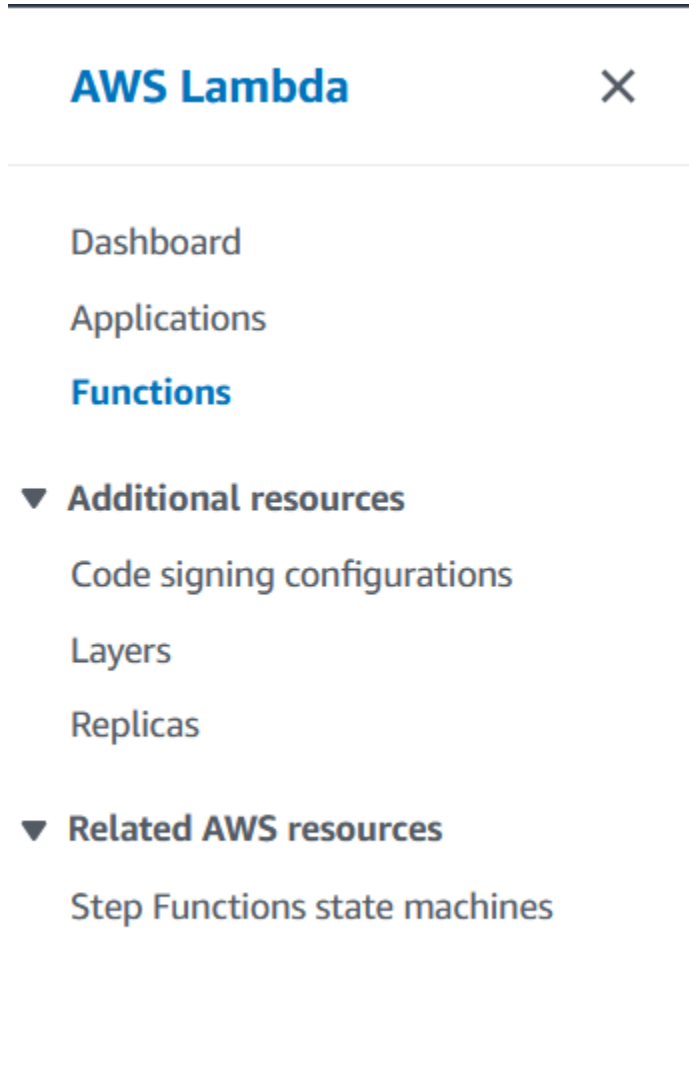
- Under expose headers and allow headers, enter the new value “content-type.” Then select the \* option under the Allow Methods header and save. The Function URL should now look like this.



- The function URL can then be used in the application to call this specific function.

## Lambda Layering

- Lambda layering is a feature from Amazon that allows you to upload libraries of code that the lambda functions can then call.
- From the main lambda page, select the option on the side to go to the layers page.



- Select create layer and choose the zip file to upload containing the libraries.

## Layer configuration

Name

*myLayerName*

Description - *optional*

*Description*

- ☒ Upload a .zip file  
☐ Upload a file from Amazon S3

 Upload

For files larger than 10 MB, consider uploading using Amazon S3.

- After selecting the runtimes and making the layer, go to the function that you want to add it to. The spot to add the layers to is at the bottom of the “code” section. Simply select “Add Layer” and then select the layer you created to add it to the function.

## Adding a new Field to the Database

- Whenever a new field is added to the database, the “add” lambda function must update to reflect this new field. The first change that needs to be made is to add the new field to the “newItem” variable. It follows the same format as every other item in it.

```
newItem = {'Recipe Name':{}, 'Directions':{},
```

- Then it needs to be added to the list of commands in the try statement below this line.  
For a “list” field, it needs to iterate through the entire list like in the example below.

```
characteristics = eventA['Characteristics']
newCharacteristics = []
for x in characteristics:
    newCharacteristics.append({'S':x})
newItem['Characteristics']['L']= newCharacteristics
```

- Strings, Booleans, and number values can be done in a single line.

```
newItem['Recipe Name']['S'] = eventA['Recipe Name']
newItem['Public']['BOOL'] = eventA['Public']
```

```
newItem['TotalTime']['N'] = str(eventA['TotalTime'])
```

- After those two changes, the function is ready for the new fields.